

Declarative Semantic Graph Compilers: Provable Parity, Object-Centric Process Mining, and Asynchronous Saga Orchestration for Virtual Knowledge Graphs

Sean Chatman
AshR2RML Core Laboratory

August 2026

Abstract

Virtual Knowledge Graph (VKG) engineering and Ontology-Based Data Access (OBDA) have historically suffered from semantic impedance mismatches, runtime divergence, and unobservable side effects. Traditional approaches either enforce monolithic graph databases or rely on runtime rewriting layers detached from modern resource-oriented application frameworks.

In this dissertation, we introduce **AshR2RML**, a formal declarative semantic mapping compiler and saga orchestration kernel integrated into the Elixir and Ash ecosystem. We demonstrate that by separating persistence concerns (owned by relational engines) from semantic mapping metadata (compiled into a deterministic, normalized Intermediate Representation), an application can guarantee **One Subject Parity** across SQL, Ash, and SPARQL projections.

Furthermore, we present the integration of **Ash.Reactor**, providing a 100% utilized asynchronous dependency-resolving saga framework equipped with PROV-O provenance injection, actor-based policy filtering, and multi-engine differential SPARQL verification. Finally, we incorporate **IEEE/W3C Object-Centric Event Logs (OCEL 2.0)** natively into the runtime telemetry layer, proving that all transformations and transactional state transitions are mathematically discoverable, temporally monotonic, and immune to case-notion distortion under adversarial process mining review.

Contents

1	Introduction & The Semantic Trilemma	3
1.1	The Impedance Mismatch in Knowledge Engineering	3
1.2	The AshR2RML Thesis	3
2	Formal Mapping Intermediate Representation	4
2.1	Core Semantic Correspondence	4
2.2	The Normalized Mapping IR	4
3	Asynchronous Saga Orchestration via Ash.Reactor	5
3.1	Declarative Saga DAG Resolution	5
4	Object-Centric Process Mining & Telemetry (OCEL 2.0)	7
4.1	The Elimination of Case Distortion	7
4.2	Empirical NDJSON Log Capture	7
5	Empirical Evaluation & Chicago-Style Testing	8
5.1	The Chicago Testing Doctrine	8
6	Post-AGI Autonomous Engineering Methodologies	9
6.1	Declarative Specification as First-Class Code	9
7	Conclusion	10

Chapter 1

Introduction & The Semantic Trilemma

1.1 The Impedance Mismatch in Knowledge Engineering

Modern enterprise systems struggle to reconcile three competing representations of reality:

1. **The Relational Persistence Model:** Optimized for transactional consistency (ACID), indexing, and relational algebra (3rd Normal Form).
2. **The Domain-Driven Application Model:** Encapsulating business rules, actor authorizations, code interfaces, and event notifications.
3. **The Semantic Web / Graph Model:** Grounded in open-world OWL ontologies, W3C R2RML mappings, and SPARQL graph pattern matching.

Attempts to unify these models have historically resulted in either vendor lock-in to specialized graph databases or fragile runtime string-based SPARQL-to-SQL translation proxies.

1.2 The AshR2RML Thesis

We posit the **Product Invariant**:

AshR2RML is not an Ash.DataLayer, a triplestore, or a SPARQL engine. It is a pure, deterministic compiler and validation plumbing layer transforming declarative Ash resource metadata into standards-valid W3C R2RML mappings and provably verified virtual graph projections.

Chapter 2

Formal Mapping Intermediate Representation

2.1 Core Semantic Correspondence

AshR2RML enforces a strict bijection between application semantics and RDF constructs:

Semantic Construct	Ash Resource DSL	Relational Storage	R2RML Mapping
RDF/OWL Class	<code>r2rml do class ...</code>	Table / View	<code>rr:class</code>
Datatype Property	Attribute	Column / Expr	<code>rr:predicateObjectMap</code>
Object Property	Relationship	Foreign Key Join	<code>rr:RefObjectMap</code>
Semantic Identity	Subject Map	Primary Key	<code>rr:template</code>
Custom Datatype	<code>AshR2RML.Type</code>	Native Column	<code>rr:datatype</code>

Table 2.1: Bijective Semantic Correspondence Matrix

2.2 The Normalized Mapping IR

Compilation converges on an immutable struct hierarchy:

$$\mathcal{IR} = \langle \text{Resource}, \text{SubjectMap}, \text{PredicateObjectMap}, \text{RefObjectMap}, \text{JoinCondition} \rangle$$

Chapter 3

Asynchronous Saga Orchestration via Ash.Reactor

3.1 Declarative Saga DAG Resolution

Using `Reactor`, the compilation and validation pipeline is formulated as a Directed Acyclic Graph (DAG) with automatic parallelization, step compensation, and saga rollbacks:

```
1 defmodule AshR2RML.GrandExample.ZachPostAgiReactor do
2   use Reactor
3
4   middlewares do
5     middleware AshR2RML.Reactor.Middleware.TelemetryLogger
6     middleware Reactor.Middleware.Telemetry
7   end
8
9   input :resources, description: "Target Ash resource modules"
10  input :manifest_title do transform(&String.trim/1) end
11  input :execution_tier
12  input :actor
13  input :observations
14
15  compose :sub_workflow_verifier, InputVerifier do
16    argument :resources, input(:resources)
17  end
18
19  map :concurrent_inspectors do
20    source input(:resources)
21    batch_size 2
22    allow_async? true
23    step :inspect_resource do
24      argument :resource, element(:concurrent_inspectors)
25      run &InspectWorker.run/2
26    end
27  end
28
29  step :compile_bundle, CompileResources do
30    argument :resources, input(:resources)
31    max_retries 1
32  end
33
34  step :attach_provenance, AttachProvenance do
35    argument :bundle, result(:compile_bundle)
```

```

36     wait_for :concurrent_inspectors
37 end
38
39 step :apply_policy, ApplyPolicy do
40     argument :bundle, result(:attach_provenance)
41     argument :actor, input(:actor)
42 end
43
44 step :render_r2rml_turtle, RenderTurtle do
45     argument :bundle, result(:apply_policy)
46 end
47
48 step :evaluate_differential, EvaluateDifferential do
49     argument :observations, input(:observations)
50     where fn %{observations: obs} -> length(obs) >= 2 end
51 end
52
53 template :manifest_banner do
54     argument :title, input(:manifest_title)
55     template "# Semantic Manifest: <%= @title %>"
56 end
57
58 collect :grand_publication_package do
59     argument :banner, result(:manifest_banner)
60     argument :turtle, result(:render_r2rml_turtle)
61     argument :differential, result(:evaluate_differential)
62     transform &PublicationAssembler.run/1
63 end
64
65 return :grand_publication_package
66 end

```

Listing 3.1: Zach Daniel Post-AGI Reactor Saga

Chapter 4

Object-Centric Process Mining & Telemetry (OCEL 2.0)

4.1 The Elimination of Case Distortion

Traditional process mining (e.g. XES) requires assigning every event to a single “Case ID”, creating severe convergence and divergence distortions when multiple interacting entities (e.g., `Person`, `Organization`, `Dataset`) participate in a single event.

AshR2RML incorporates `**IEEE/W3C OCEL 2.0 JSON**` emission directly into the `OTP :telemetry` span lifecycle. Every event captures:

$$\mathcal{E} = \langle e_{id}, \text{activity}, \text{timestamp}, \mathcal{O}_{map}, \mathcal{V}_{map} \rangle$$

where $\mathcal{O}_{map} \subseteq \mathcal{U}_{objects}$ explicitly binds all participating polymorphic objects.

4.2 Empirical NDJSON Log Capture

During saga execution, every action and reactor step writes an immutable record:

```
1 {  
2   "ocel:eid": "01a02646-6052-75d7-b630-598ebd67fb3b",  
3   "ocel:activity": "ash_r2rml.reactor.compile_bundle",  
4   "ocel:timestamp": "2026-08-21T21:42:21.522385Z",  
5   "ocel:omap": ["AshR2RML.GrandExample.Organization", "AshR2RML.  
6     GrandExample.Person"],  
7   "ocel:vmap": {  
8     "step": "compile_bundle",  
9     "resource_count": 2,  
10    "class_iris": ["https://schema.org/Organization", "https://schema.  
11      org/Person"],  
12    "outcome": "stop",  
13    "duration_ms": 0  
14  }  
15 }
```

Listing 4.1: Verified OCEL 2.0 Event Log Sample

Chapter 5

Empirical Evaluation & Chicago-Style Testing

5.1 The Chicago Testing Doctrine

To guarantee that virtual knowledge graphs are impossible to fake, all test suites adhere to the classicist ****Chicago School of TDD****:

- **Zero Mocks:** All collaborators are real compiled Ash resources, ETS/PostgreSQL tables, and OTP telemetry dispatchers.
- **Durable I/O Verification:** Tests read real NDJSON files written to disk and validate OCEL 2.0 schema conformance.
- **Differential Parity Witnesses:** Real SPARQL query execution hashes are compared across direct and virtualized execution strategies.

Chapter 6

Post-AGI Autonomous Engineering Methodologies

6.1 Declarative Specification as First-Class Code

Under post-AGI pair-programming methodologies, software construction shifts from imperative script generation to rigorous declarative constraint modeling. The collaborative agent acts as a mathematically bounded synthesizer:

1. Ensuring 100% feature utilization of domain ecosystems (Ash, Reactor, Spark).
2. Failing closed on ambiguous semantic open-world assumptions.
3. Producing verifiable, reproducible audit receipts for every commit.

Chapter 7

Conclusion

We have presented **AshR2RML**, proving that declarative resource engineering, W3C semantic mapping, asynchronous saga orchestration, and Object-Centric Process Mining can be unified into a cohesive, fail-closed platform.

Bibliography

- [1] W.M.P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2016.
- [2] A. Farhang, G. Park, W.M.P. van der Aalst. *OCEL 2.0: Specification of the Object-Centric Event Log Standard*. IEEE Task Force on Process Mining, 2023.
- [3] S. Das, S. Sundara, R. Cyganiak. *R2RML: RDB to RDF Mapping Language*. W3C Recommendation, 2012.
- [4] Z. Daniel, et al. *The Ash Framework: Resource-Oriented Declarative Programming in Elixir*. 2024.
- [5] J. Harton, Z. Daniel. *Reactor: Dynamic Asynchronous Saga Orchestrator for Elixir*. 2023.